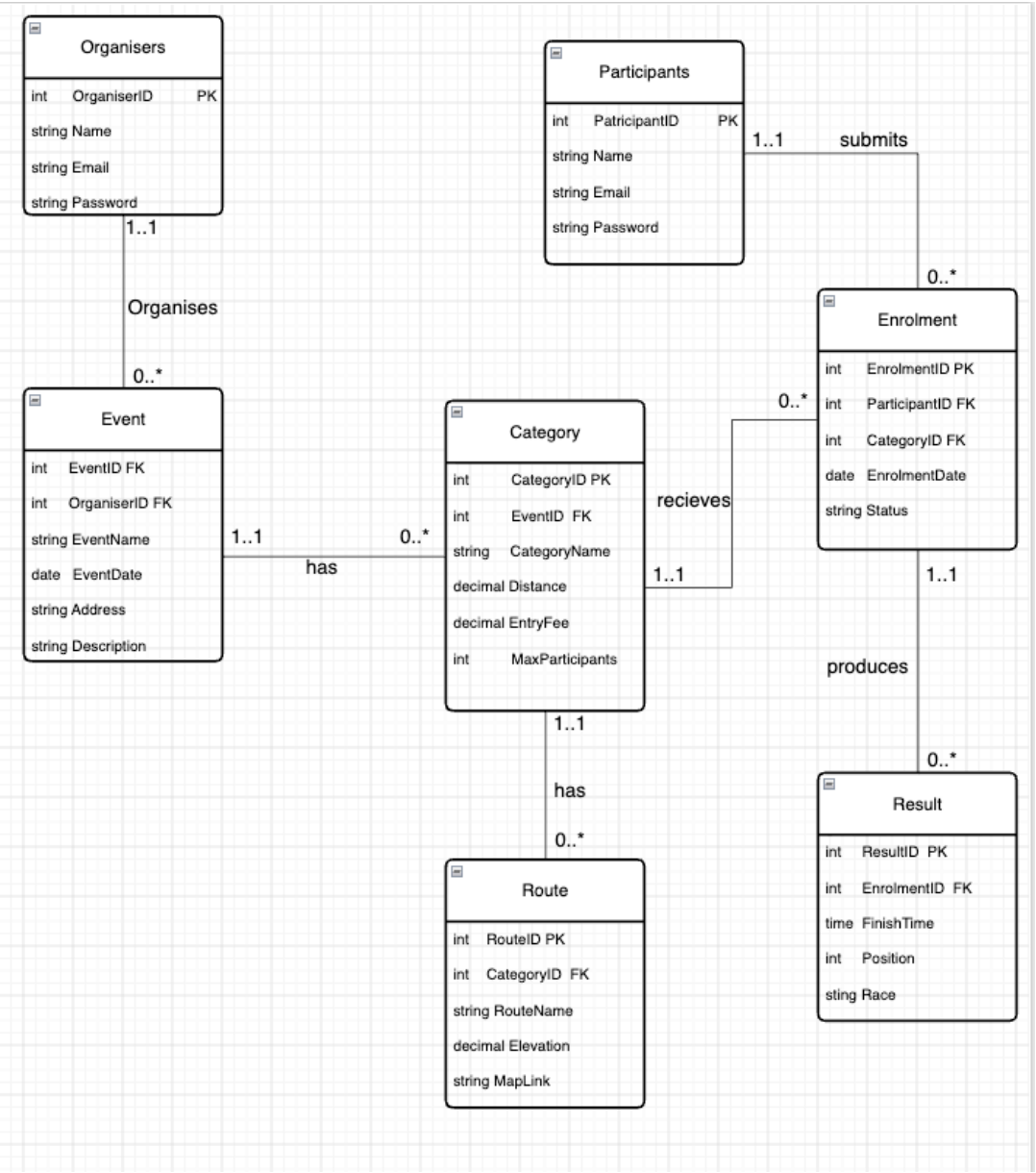
Section A : Raceday ERD

Part 1 : Section A



Our ERD above is a representation of our database for the RaceDay web application system. We have been able to draw this up using the business rules given to us in the scenario. The idea behind mapping out this ERD is to understand the main actors in the system.

Who ? That is the first question. We have two main roles in the scenario which is the Organizer and the participants. This gives us our first two entities and an idea of how to decide on the appropriate attributes for this entity. Then we need to consider what these participants are able to do. Organizers are able to set events these events are broken into categories and these categories have their own routes. All this information provides us with the rest of 3 entities for our ERD. Events categories and routes are all connected through the enrollment activity which is the last entity that participants interact with to connect to the activities setup by the organizer.

Primary Keys have been created for each entity in the ERD coming to a total of 7 Primary Keys. The Foreign Keys have been created for entity tables that reference other tables in the database coming to a total of 6 Foreign Keys.

Entities for the ERD are as follows:

- Organizer - (OrganizerID {PK} UserName Email Password)
- Event - (EventID {PK} OrganizerID {FK} EventName EventDate Address Description)
- Category - (CategoryID {PK} EventID {FK} CategoryName Distance EntryFee MaxParticipants)
- Participants - (ParticipantID {PK} Name Email Password)
- Enrollment - (EnrollmentID {PK} ParticipantID {FK} CategoryID {FK} EnrollmentDate Status)
- Result - (ResultID {PK} EnrollmentID {FK} FinishTime Position Race)
- Route - (RouteID {PK} CategoryID {FK} RouteName Elevation MapLink)

The cardinality pattern that runs through the whole diagram

Once the entities were settled, every relationship follows the same shape: **one parent, many optional children.**

- One Organizer → many Events (0..*, an Organizer can have none yet)
- One Event → many Categories
- One Category → many Routes
- One Category → many Enrollments
- One Participant → many Enrollments
- One Enrollment → many Results

RaceDay — API Endpoint Plan

Part 1, Section B — updated for the split Organizer/Participant schema (Organizer, Participant, Event, Category, Route, Enrollment, Result). Registration is now role-specific since there's no shared table to insert into; login stays unified since it only needs to find a matching email/password across both tables.

Table 1 — Participant / user-facing endpoints

Covers registration, login, profile, searching/browsing events, entering (enrolling in) a category, and tracking personal results.

Method	Route	Description	Request Body	Use Case	Expected Response
POST	/api/auth/register/participant	Creates a new row in the Participant table	{ "name", "email", "password" }	A new runner signs up before entering their first race	201 Created → { "participantId", "token" }
POST	/api/auth/login	Checks both Organiser and Participant tables for a matching email/password; returns a token carrying the role found	{ "email", "password" }	A returning Participant logs in to check their upcoming races	200 OK → { "token", "role": "Participant" }
GET	/api/participants/me	Views own profile from the Participant table	—	A Participant checks the email/name linked to their account	200 OK → Participant object
PUT	/api/participants/me	Updates own profile details in the Participant table	{ "name"?, "email"?, "password"? }	A Participant updates their contact details before race day	200 OK → updated Participant object
GET	/api/events?search=&location=&date=	Searches/browses upcoming events by keyword, location, or date	—	A Participant searches "10km Johannesburg October" to find a race	200 OK → array of Event objects
GET	/api/events/{eventId}	Views full details of one event	—	A Participant reads the event description and rules before entering	200 OK → Event object
GET	/api/events/{eventId}/categories	Views the categories available for an event	—	A Participant compares the 5km and 10km options	200 OK → array of Category objects
POST	/api/enrolments	Enters (enrols in) a category; ParticipantID is taken from the logged-in token, not the request body	{ "categoryId": int }	A Participant enters the 10km category by entering their details and confirming	201 Created → new Enrolment object
GET	/api/enrolments/me	Lists own current enrolments	—	A Participant checks which races they're entered in	200 OK → array of Enrolment objects

Method	Route	Description	Request Body	Use Case	Expected Response
DELETE	/api/enrolments/{enrolmentId}	Withdraws own enrolment (must own it)	—	A Participant pulls out of a race due to injury	204 No Content
GET	/api/results/me	Lists own result history	—	A Participant tracks their personal-best times over time	200 OK → array of Result objects
GET	/api/events/{eventId}/results	Views the public leaderboard for an event	—	A Participant checks how they placed after race day	200 OK → array of Result objects

Table 2 — Organiser / admin CRUD endpoint

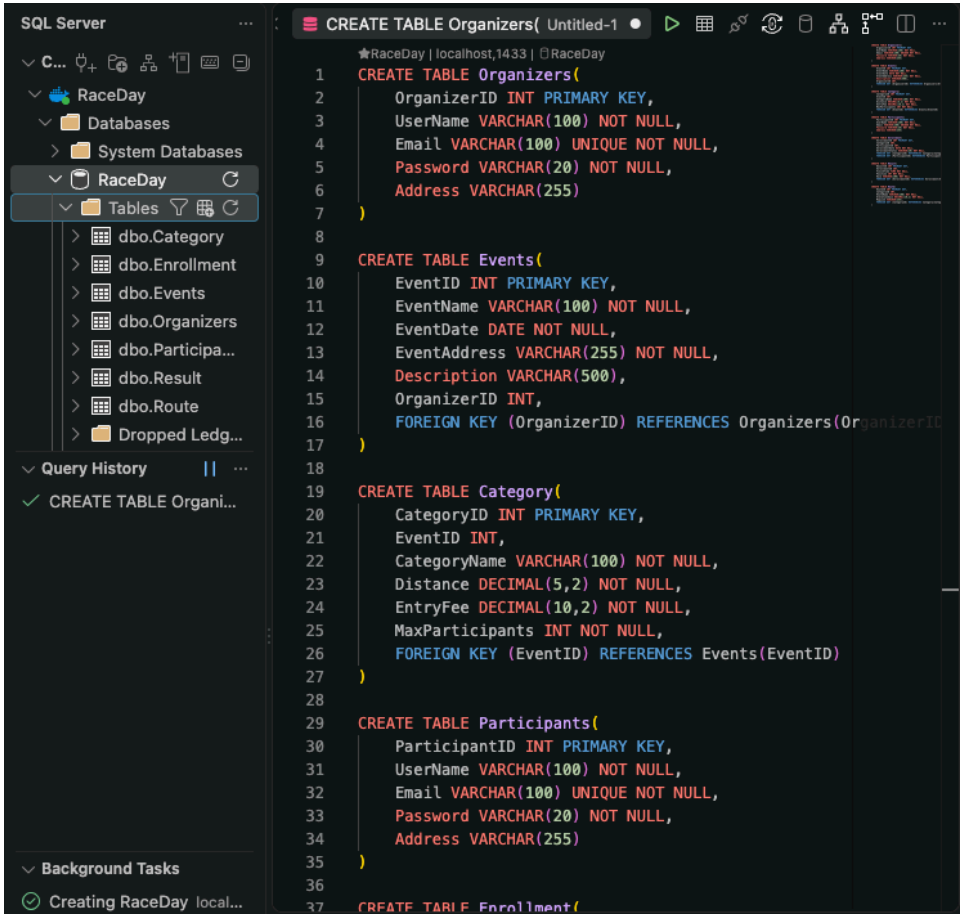
Full create, read, update, delete control over events, categories, and results.

Method	Route	Description	Request Body	Use Case	Expected Response
POST	/api/auth/register/organiser	Creates a new row in the Organiser table	{ "name", "email", "password" }	A club administrator signs up to start hosting races	201 Created → { "organiserId", "token" }
POST	/api/auth/login	Checks both Organiser and Participant tables for a matching email/password; returns a token carrying the role found	{ "email", "password" }	An Organiser logs in to manage their events	200 OK → { "token", "role": "Organiser" }
GET	/api/organisers/me	Views own profile from the Organiser table	—	An Organiser checks the email linked to their account	200 OK → Organiser object
PUT	/api/organisers/me	Updates own profile details in the Organiser table	{ "name"?, "email"?, "password"? }	An Organiser updates their contact details	200 OK → updated Organiser object
POST (Create)	/api/events	Creates a new event; OrganiserID is taken from the logged-in token	{ "eventName", "eventDate", "address", "description" }	An Organiser sets up a new road race	201 Created → new Event object
GET (Read)	/api/events/organiser/me	Lists all events created by the logged-in Organiser	—	An Organiser views a dashboard of their own events	200 OK → array of Event objects
PUT (Update)	/api/events/{eventId}	Edits an event (must own it)	{ "eventName"?, "eventDate"?, "address"?, "description"? }	An Organiser changes the venue after a route closure	200 OK → updated Event object
DELETE	/api/events/{eventId}	Deletes an event (must own it)	—	An Organiser cancels an event due to bad weather	204 No Content
POST (Create)	/api/events/{eventId}/ categories	Adds a category to an event	{ "categoryName", "distance", "entryFee", "maxParticipants" }	An Organiser adds a 21km category	201 Created → new Category object
PUT (Update)	/api/categories/{categoryId}	Edits a category (must own parent event)	{ "categoryName"?, "distance"?, "entryFee"?, "maxParticipants"? }	An Organiser raises the entry fee	200 OK → updated Category object
DELETE	/api/categories/{categoryId}	Removes a category (must own parent event)	—	An Organiser drops an under-subscribed category	204 No Content
POST (Create)	/api/categories/{categoryId}/ routes	Adds a route to a category	{ "routeName", "elevation", "mapLink" }	An Organiser uploads the official route map for the 10km	201 Created → new Route object
GET (Read)	/api/events/{eventId}/ enrolments	Lists every enrolment for an event, all categories	—	An Organiser checks headcount per category before race day	200 OK → array of Enrolment objects
POST (Create)	/api/results	Captures a result for a completed enrolment	{ "enrolmentId", "finishTime", "position", "race" }	An Organiser captures finish times after the race	201 Created → new Result object
PUT (Update)	/api/results/{resultId}	Corrects a captured result (must own parent event)	{ "finishTime"?, "position"?, "race"? }	An Organiser fixes a timing error spotted after entry	200 OK → updated Result object

Method	Route	Description	Request Body	Use Case	Expected Response
DELETE	/api/results/{resultId}	Removes an incorrectly captured result (must own parent event)	—	An Organiser deletes a duplicate result entry	204 No Content

Section C : SQL Script

The Create Table SQL Statements responsible for creating the tables associated with the ERD and our database as whole. With all 7 primary keys defined for each entities and the foreign keys as well. Foreign keys are responsible for linking two different tables in a database. The Primary Key is responsible for making a record uniquely identifiable. Below we have created 7 tables.



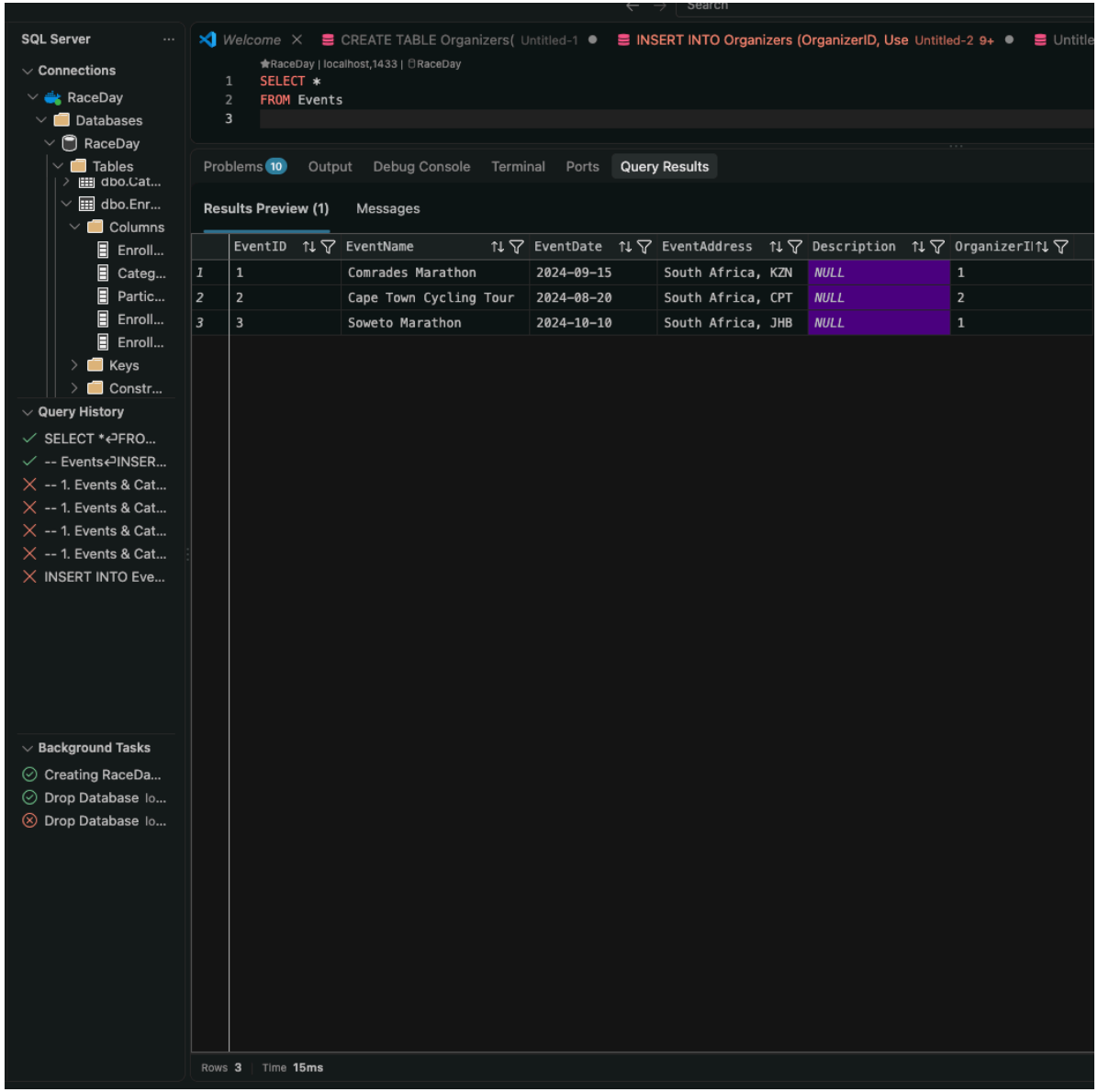

```
CREATE TABLE Enrollment(
    EnrollmentID INT PRIMARY KEY,
    CategoryID INT,
    ParticipantID INT,
    EnrollmentDate DATE NOT NULL,
    EnrollmentStatus VARCHAR(20) NOT NULL,
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (ParticipantID) REFERENCES Participant
)

CREATE TABLE Result(
    ResultID INT PRIMARY KEY,
    EnrollmentID INT,
    FinishTime TIME NOT NULL,
    Position INT NOT NULL,
    Race VARCHAR(100) NOT NULL,
    FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(E
)

CREATE TABLE Route(
    RouteID INT PRIMARY KEY,
    CategoryID INT,
    RouteName VARCHAR(100) NOT NULL,
    ElevationGain DECIMAL(10,2) NOT NULL,
    MapLink VARCHAR(255),
    FOREIGN KEY (CategoryID) REFERENCES Category(Categ
)
```

The INSERT SQL statement that we use to populate the tables that we have created in the database. These tables have been defined with data types that our INSERT values must now adhere to when being populated into a particular table.

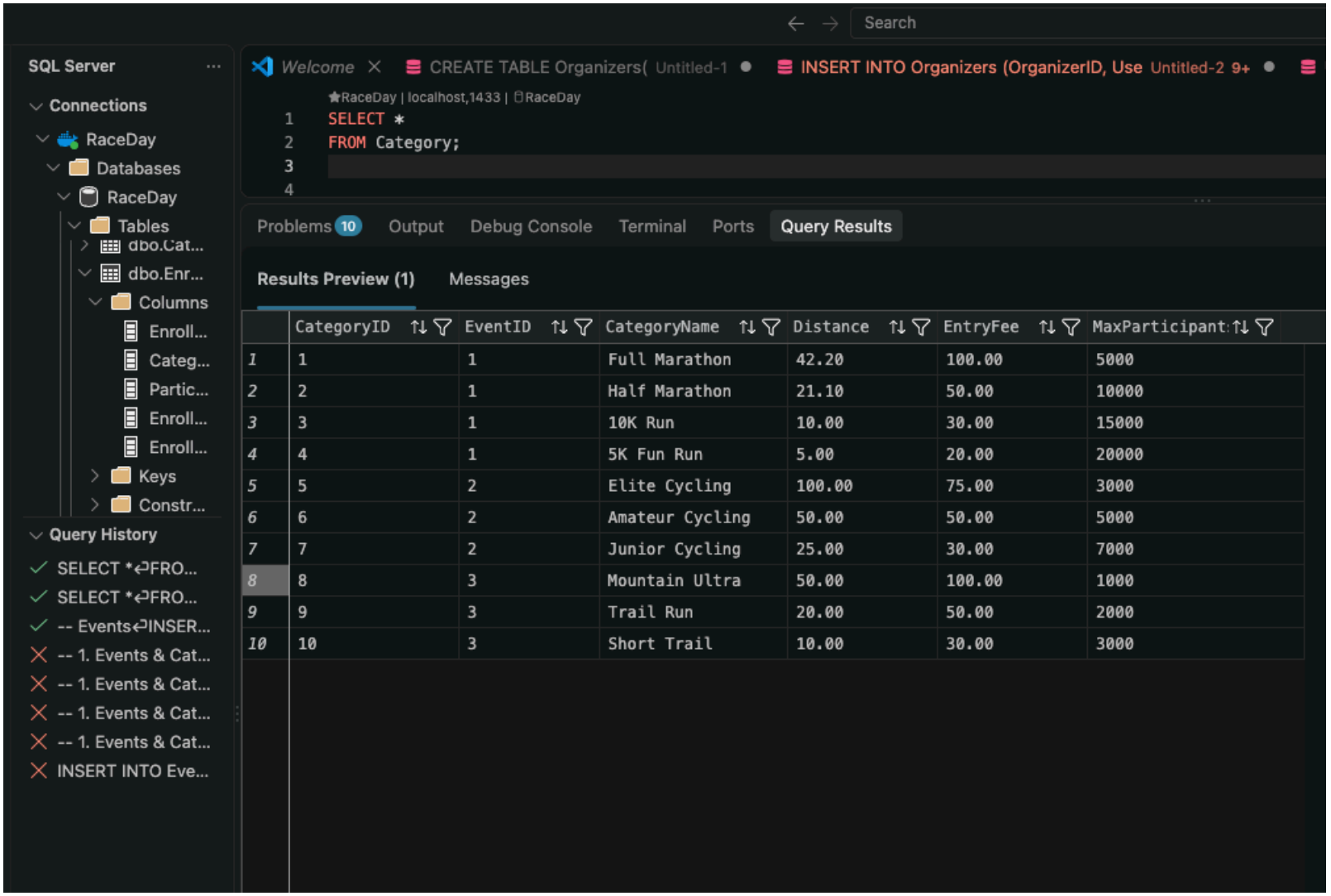

```sql
-- Events
INSERT INTO dbo.Events
    (EventID, EventName, OrganizerID, EventDate, EventAddress)
SELECT *
FROM (VALUES
    (1, 'Comrades Marathon', 1, CAST('2024-09-15' AS date), 'South Africa, KZN'),
    (2, 'Cape Town Cycling Tour', 2, CAST('2024-08-20' AS date), 'South Africa, CPT'),
    (3, 'Soweto Marathon', 1, CAST('2024-10-10' AS date), 'South Africa, JHB')
) AS NewEvents(EventID, EventName, OrganizerID, EventDate, EventAddress)
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Events
    WHERE Events.EventID = NewEvents.EventID
);

-- Categories
INSERT INTO dbo.Category
    (CategoryID, EventID, CategoryName, Distance, EntryFee, MaxParticipants)
SELECT *
FROM (VALUES
    (1, 1, 'Full Marathon', 42.195, 100.00, 5000),
    (2, 1, 'Half Marathon', 21.0975, 50.00, 10000),
    (3, 1, '10K Run', 10.000, 30.00, 15000),
    (4, 1, '5K Fun Run', 5.000, 20.00, 20000),
    (5, 2, 'Elite Cycling', 100.000, 75.00, 3000),
    (6, 2, 'Amateur Cycling', 50.000, 50.00, 5000),
    (7, 2, 'Junior Cycling', 25.000, 30.00, 7000),
    (8, 3, 'Mountain Ultra', 50.000, 100.00, 1000),
    (9, 3, 'Trail Run', 20.000, 50.00, 2000),
    (10, 3, 'Short Trail', 10.000, 30.00, 3000)
) AS NewCategories
    (CategoryID, EventID, CategoryName, Distance, EntryFee, MaxParticipants)
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Category
    WHERE Category.CategoryID = NewCategories.CategoryID
);

-- Enrollments
INSERT INTO dbo.Enrollment
    (EnrollmentID, ParticipantID, CategoryID, EnrollmentDate, EnrollmentStatus)
SELECT *
FROM (VALUES
    (1, 1, 1, CAST('2024-07-01' AS date), 'Confirmed'),
    (2, 2, 2, CAST('2024-07-02' AS date), 'Confirmed'),
    (3, 1, 3, CAST('2024-07-03' AS date), 'Pending'),
    (4, 2, 4, CAST('2024-07-04' AS date), 'Confirmed'),
    (5, 1, 5, CAST('2024-07-05' AS date), 'Confirmed'),
    (6, 2, 6, CAST('2024-07-06' AS date), 'Pending'),
    (7, 1, 7, CAST('2024-07-07' AS date), 'Confirmed'),
```

In the snapshot above we show how we used the INSERT statement to populate the events table in accordance with the appropriate events provided in the business scenario. We have 3 Events all with their own EventID to uniquely identify events and make data capturing easier. We also populated the Category table with 10 categories of races that are available for participants to enter on the RaceDay. We have also been tasked with providing the table with sample enrollment data to show how the table should be populated making use of INSERT statement.

Events Table

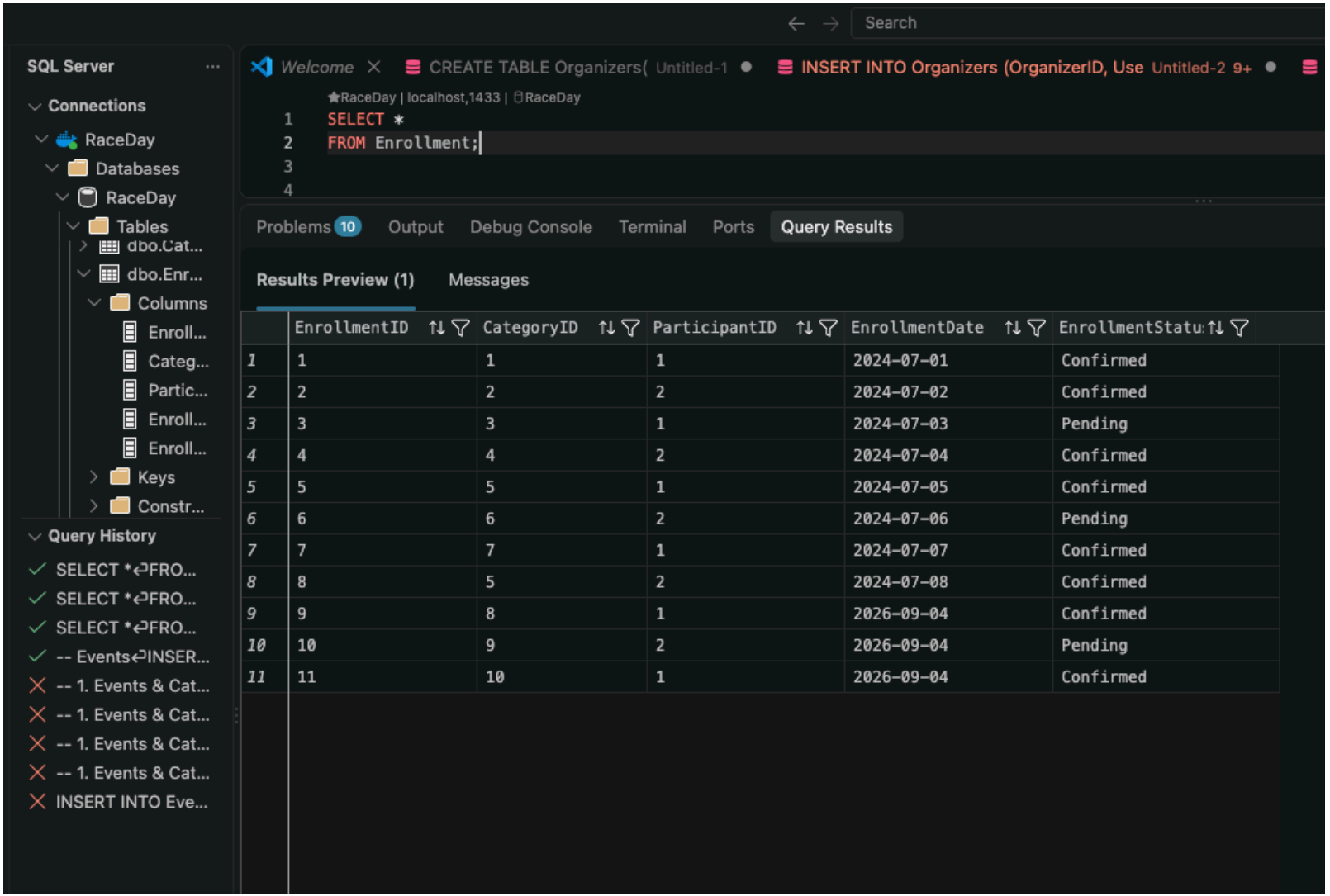


The Category Table



	CategoryID	EventID	CategoryName	Distance	EntryFee	MaxParticipant
1	1	1	Full Marathon	42.20	100.00	5000
2	2	1	Half Marathon	21.10	50.00	10000
3	3	1	10K Run	10.00	30.00	15000
4	4	1	5K Fun Run	5.00	20.00	20000
5	5	2	Elite Cycling	100.00	75.00	3000
6	6	2	Amateur Cycling	50.00	50.00	5000
7	7	2	Junior Cycling	25.00	30.00	7000
8	8	3	Mountain Ultra	50.00	100.00	1000
9	9	3	Trail Run	20.00	50.00	2000
10	10	3	Short Trail	10.00	30.00	3000

The Enrollment Table



	EnrollmentID	CategoryID	ParticipantID	EnrollmentDate	EnrollmentStatu
1	1	1	1	2024-07-01	Confirmed
2	2	2	2	2024-07-02	Confirmed
3	3	3	1	2024-07-03	Pending
4	4	4	2	2024-07-04	Confirmed
5	5	5	1	2024-07-05	Confirmed
6	6	6	2	2024-07-06	Pending
7	7	7	1	2024-07-07	Confirmed
8	8	5	2	2024-07-08	Confirmed
9	9	8	1	2026-09-04	Confirmed
10	10	9	2	2026-09-04	Pending
11	11	10	1	2026-09-04	Confirmed